PROGRAMACIÓN IV

Trabajo Práctico - CRUD Categorías + CRUD Productos

OBJETIVO GENERAL

Desarrollar una aplicación Full Stack conectando un frontend React + TypeScript con un backend FastAPI. El estudiante implementará el CRUD completo de Categorías desde la interfaz de usuario, integrando el backend mediante fetch nativo. Los CRUDs de Producto y ProductoCategoria serán construidos en el backend y demostrados mediante un archivo REST Client.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
React	Biblioteca para construir interfaces de usuario mediante componentes reutilizables.
TypeScript	Superset tipado de JavaScript para detectar errores en compilación y mejorar la legibilidad.
Vite	Herramienta de build moderna que inicializa proyectos React con TypeScript, reemplazando Create React App.
FastAPI	Framework Python de alto rendimiento para construir APIs REST. Permite definir endpoints, validar datos con Pydantic y exponer documentación automática en /docs.
Pydantic	Librería de validación de datos usada por FastAPI para definir modelos (schemas) con tipado estricto.
fetch nativo	API del navegador para realizar peticiones HTTP. Se usa con async/await para consumir la API REST desde el frontend.
useState	Hook de React para manejar estado local en un componente funcional (ej: lista de categorías, estado del modal).
useEffect	Hook de React para ejecutar efectos secundarios (ej: fetch al montar el componente para cargar datos).
Interface	Construcción de TypeScript que define la forma de un objeto (ej: Categoria, Producto).
Tailwind CSS	Framework de CSS utilitario. Estilos aplicados directamente en el JSX mediante clases predefinidas.
CORS	Mecanismo que permite al frontend (localhost:5173) comunicarse con el backend (localhost:8000) en dominios distintos.

CASO PRÁCTICO

Parte A — Backend con Fast Api

Tomando como base el trabajo práctico de la unidad 3:

a) Crear y activar un entorno virtual Python, crea archivo de dependencias e instalar las dependencias:

```
python -m venv .venv
source .venv/bin/activate   # macOS/Linux
.venv\Scripts\activate     # Windows
pip install -r requirements.txt
```

b) Definir los models y schemas del modulo Catálogo de Productos restantes.
(SVG adjunto a este pdf)

c) Al módulo “Catálogo de productos” agregar campos faltantes (Auditoría)
d) Crear un CRUD siguiendo la arquitectura **Models-Schemas-Service-Router**

Notas:

Recuerda agregar CORS para poder consumir tu API desde su frontend

e) Los endpoints deberán de tener la optimización aprendida en la unidad y respuestas estandarizadas

f) Ejecutar el servidor:

```
python -m fastapi dev main.py
```

📘 Parte B — Frontend con React + TypeScript (MEJORADO)

g) Páginas (Routing)

La aplicación debe estar organizada en páginas separadas.

Crear en src/pages/:

- ***CategoriasPage.tsx***
- ***ProductsPage.tsx***

CategoriasPage

Debe contener:

- *Toda la lógica de categorías:*
  - *useQuery*
  - *useMutation*
  - *manejo del modal*
- *Renderiza:*
  - *CategoriaList*
  - *CategoriaModal*
  - *botón "Nueva categoría"*

ProductsPage

- *Estructura similar a CategoriasPage*
- *Preparada para trabajar con productos*
- *Puede reutilizar lógica (recomendado)*
- *Debe tener:*
  - *listado*
  - *formulario (modal o página)*

h) Navegación

Implementar rutas usando React Router.

Instalar:

pnpm add react-router-dom

Configuración básica

Definir rutas:

- */categorias → CategoriasPage*
- */productos → ProductsPage*

Navbar

Actualizar Navbar.tsx para incluir navegación:

- *Link a **Categorías***
- *Link a **Productos***

i) App.tsx (cambio clave)

*Ahora **App ya no tiene lógica de negocio**.*

Solo debe encargarse de:

- *Definir rutas*
- *Renderizar el Navbar*
- *Mostrar las páginas*

Comportamiento esperado (actualizado)

- *La app inicia en /categorias*
- *Se puede navegar a /productos*
- *Cada página maneja su propia lógica*
- *No hay lógica mezclada en App (orden profesional)*

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Integración Full Stack:** El frontend React consume la API FastAPI mediante fetch nativo. El CRUD de Categorías funciona de extremo a extremo.
- **Modal con useState:** El formulario de alta y edición está dentro de un modal controlado con estado booleano.
- **Hooks aplicados correctamente:** useState y useEffect utilizados en App.tsx. Los demás componentes son funcionales puros sin hooks.
- **Tipado TypeScript:** Interfaces Categoria y Producto definidas. Props tipados en todos los componentes .tsx.
- **Backend REST documentado:** Endpoints de Producto y ProductoCategoria funcionando y demostrados en el archivo .http.
- **CORS configurado:** El backend permite peticiones desde el origen del frontend.

RÚBRICA DE EVALUACIÓN

Criterio	Descripción	Puntos
CRUD Categorías – Frontend	Listar, crear, editar y eliminar categorías desde React usando fetch nativo y useState/useEffect.	30
Modal funcional	Modal que se abre para crear y editar categorías, controlado con useState. Se cierra al guardar o cancelar.	15
CRUD Categorías – Backend	Endpoints FastAPI GET /categorias, POST, PUT/{id}, DELETE/{id} con Pydantic y persistencia en lista/DB.	20
CRUD Productos y relación – REST Client	Endpoints de Productos y ProductoCategoria demostrados con REST Client (.http). No requiere UI.	15
Tipado TypeScript	Interfaces Categoria y Producto tipadas. Props tipados en todos los componentes.	10
Estructura y buenas prácticas	Carpetas components/ y types/. Tailwind CSS aplicado. Sin errores de TS en consola.	10

Modalidad de Entrega

- Eliminar la carpeta node_modules antes de comprimir
- Proyecto Frontend comprimido en un archivo .zip
- Proyecto Backend comprimido en un archivo .zip

RECURSOS ADICIONALES

📚 **Documentación oficial:**

- React: https://react.dev/
- TypeScript: https://www.typescriptlang.org/docs/
- Tailwind CSS: https://tailwindcss.com/docs/
- Vite: https://vitejs.dev/
- FastAPI: https://fastapi.tiangolo.com/